

COSC 6385 - Homework 1: Branch Prediction

Nicholas Anderson

1 Problem Statement

For this homework we have been tasked with creating a branch prediction class in c++. Template code was given, of which the important methods are its ability to read instructions and identify branch statements. We are given a multitude of different traces, and by using the *run* command we can find the average mispredictions-per-1000-instructions (MPKI) across all given traces.

We were tasked with implementing *branch outcome prediction* using a two-way global predictor, and a bimodal predictor. We were not required to implement loop prediction or branch target prediction. Essentially, we use a bimodal table, and a two-way global predictor to predict whether a branch will be true or not. Additionally, implementation of the path information register (PIR) and the hash access function was not required for the global predictor.

To do this we make modifications to the **pm_predictor** class located in **my_predictor.h** header file. Though as I will discuss later, I found it useful to modify other portions of the code as well for my implementation.

2 Preliminary Results

Initially, the prediction is set to simply always predict that a branch is not taken. This results in an average MPKI of 86.311 across all given test data.

3 My Solution

My code can be found at [1], which is the github repository containing my homework solutions.

1. Implement bimodal prediction, of which there is no ambiguity in the problem statements
2. Implement global prediction, of which there is an ambiguity of the replacement policy, and whether the use of the a global history vector is required. I used LRU as my replacement policy, and no global history vector
3. The branch prediction is based off of only one prediction, depending on whether there is a hit in the global prediction buffer or not. For this reason, the bimodal 2bc is only updated on a global prediction buffer miss. The global prediction buffer 2bc is always updated, since on a miss a tag is replaced and the counter is reset to either weakly taken or weakly not taken.

References

- [1] Nick123293. cosc-6385-computer-architecture. <https://github.com/Nick123293/cosc-6385-Computer-Architecture>, 2026. Accessed: 10-8-2026.